UNDERSTANDING OOP CONCEPTS (CLASSES, OBJECTS, INHERITANCE, ENCAPSULATION, POLYMORPHISM)

What is OOP? OOP stands for Object-Oriented Programming.

Object-Oriented Programming (OOP) is a programming paradigm that allows you to model real-world entities as objects, enabling you to organize and structure your code in a more modular and reusable way.

OOP is designed in such a way that one should focus on an object while programming and not the procedure.

Concepts of OOP

Let us try to understand a little about all these concepts, through a simple examples

Classes and Objects:

Class: A class is a blueprint or template for creating objects. It defines the properties (attributes) and behaviors (methods) that the objects of that class will have. Here we can take Human Being as a class. Human Being, having body parts, and performing various actions.

Inheritance: Inheritance allows you to create a new class (subclass or derived class) based on an existing class (superclass or base class). The subclass inherits properties and methods from the superclass. Considering HumanBeing a class, which has properties like hands, legs, eyes etc, and functions like walk, talk, eat, see etc. Male and Female are also classes, but most of the properties and functions are included in HumanBeing, hence they can inherit everything from class HumanBeing using the concept of Inheritance.

Object: An object is an instance of a class. It represents a specific entity with its own set of data and behaviors. My name is Abdul, and I am an instance/object of class Male. When we say, Human

Being, Male or Female, we just mean a kind, you, your friend, me we are the forms of these classes. We have a physical existence while a class is just a logical definition. We are the objects.

Abstraction

Abstraction means, showcasing only the required things to the outside world while hiding the details. Continuing our example, Human Being's can talk, walk, hear, eat, but the details are hidden from the outside world. We can take our skin as the Abstraction factor in our case, hiding the inside mechanisms.

Encapsulation: Encapsulation refers to the practice of bundling data (attributes) and methods (functions) that operate on that data into a single unit (class). It helps in controlling access to data and protecting it from unintended modification.

Polymorphism: Polymorphism allows objects of different classes to be treated as objects of a common superclass. This enables you to write code that can work with objects of multiple types without knowing their specific class.

These OOP concepts help in creating organized, modular, and maintainable code by promoting reusability, abstraction, and a clearer representation of real-world entities in your programming projects.

Object-oriented programming has several advantages over procedural programming:

- OOP is faster and easier to execute
- OOP provides a clear structure for the programs
- OOP helps to keep the PHP code DRY "Don't Repeat Yourself", and makes the code easier to maintain, modify and debug

- OOP makes it possible to create full reusable applications with less code and shorter development time

Tip: The "Don't Repeat Yourself" (DRY) principle is about reducing the repetition of code. You should extract out the codes that are common for the application, and place them at a single place and reuse them instead of repeating it.

CREATING AND USING CLASSES AND OBJECTS IN PHP

Defining a Class in PHP

When we define a class in PHP, just like any variable we have to give our class a name, and there are some rules that we must follow while naming our class in PHP, they are:

- The class name should start with an alphabet.
- The class name should not be a reserved PHP keyword.
- The class name should not contain any spaces.

Now that we know the rules related to naming convention for classes in PHP, let's create a simple class with name **Person**.

Syntax for defining a Class in PHP

The syntax for defining a class in PHP is very simple, we use the keyword `class` followed by the **name of the class** and then we enclose the code for the class within **curly braces { }** just like for a function/method.

Below is the code for class `Person`:

```php
<?php
    class Person {

        // class variables and methods comes here...

    }
?>
```

Although it is not mandatory, but it is good practice to use the class name as the filename for the PHP file. Also, PHP allows for multiple classes to be defined in a single file, but again it is not recommended.

So we will save the above code for the class `Person` in the file **Person.php**.

Now let's add a variable to our class to store **name** of a person. When we create a variable inside a class, it is called **property**.

```php
<?php
    class Person {
        // to store name of person
        var $name;

    }
?>
```

`var` is the keyword to define a variable/property inside a class and `$name` is the name of the variable. It's time to add methods/functions inside our class. Functions, when defined inside a class are called **methods**.

Inside a class, methods are defined to either perform some operation on the class variables (properties) or perform some other operation for the class, like print the values of all the variables, or may be store data into database etc.

When we define methods to **get** the value of a class variable and to **set** value of a class variable, these methods are called **Getter and Setter** functions. These are popularly named as `get_NAME-OF-VARIABLE()` and `set_NAME-OF-VARIABLE()`.

So, for the variable `$name`, let's add `get_name()` and `set_name()` functions.

```php
<?php
    class Person {
        // to store name of person
        var $name;

        // print name
        function get_name() {
            return $this->name;
        }

        function set_name($new_name) {
            $this->name = $new_name;
        }

    }
?>
```

We will talk more about why we need getter and setter functions, when we will learn how to create objects of class.

Can we define a Class without any Variable?

Yes, a simple class does not require any variables. We can simply have some methods in it. Such classes are generally helper class with some useful common methods in them.

Let's create a simple class with name **Human** with two methods:

```php
<?php
    class Human {

        // for male
        function male() {
            echo "This human is a male";
        }

        function female() {
            echo "This human is a female";
        }

    }
?>
```

For the above class, all the objects will be similar as there is no variable.

When we have variables in a class, we can create multiple objects of that class with different values for the class variables.

So by now, we hope you have understood what a class is, its nothing but a user defined data type, which has properties/variables and class methods. In the next topic we will learn about how to create class objects.

PHP CLASS OBJECT

As discussed a class is nothing but a blueprint or a user defined datatype which can be used by creating its objects. Class is a logical entity while its object is real. A **class** generally includes local variables and local methods. And an **object** is an instance of the class which holds the local variables with values assigned and using the object we can call the local methods defined in the class.

To create an object of a class we use the `new` keyword.

```php
<?php
    class Studytonight {
        // to store name of person
        var $url = "studytonight.com";

        // simple class method
        function desc() {
            echo "Studytonight helps you learn Coding.";
        }
    }

    // creating class object
    $obj = new Studytonight();
?>
```

As you can see, we used the new keyword in the last line to create an object of the class Studytonight and assigned it to the variable $obj. Once we have an object of any class, we can use it to access the class methods and variable using -> operator

```php
<?php
    class Studytonight {
        // variable with a default value
        var $url = "studytonight.com";

        // simple class method
        function desc() {
            echo "Studytonight helps you learn Coding.";
        }
    }

    // creating class object
    $obj = new Studytonight();
    // accessing class variable
    echo $obj->url . "<br/>";
    // calling class method
    $obj->desc();
?>
```

Output:

```
OUTPUT:

studytonight.com

Studytonight helps you learn Coding.
```

Accessing Class Variables and Methods

To access class variable and methods using the object of the class, we use the -> operator followed by the name of the variable or the method's name.

We have already showcased this in the example above, but let's have another example, with a little more complex class.

```php
<?php
    class Person {
        // first name of person
        var $fname;
        // last name of person
        var $lname;

        function showName() {
            echo "My name is: " . $this->fname . " " . $this->lname;
        }
    }

    // creating class object
    $john = new Person();
    // assigning values to variables
    $john->fname = "John";
    $john->lname = "Wick";
    // calling the method function
    $john->showName();
```

OUTPUT:

```
My name is: John Wick
```

In the above class, inside the `showName()` method we have used `$this` keyword, it is used to refer the calling object. We will learn more about it in upcoming tutorials.

Similarly we can create multiple objects of a class with different value assigned to the local class variables. Let's do that

```php
<?php
    2 references | 0 implementations
    class Person {
        // first name of person
        3 references
        var $fname;
        // last name of person
        3 references
        var $lname;

        2 references | 0 overrides
        function showName() {
            echo "My name is: " . $this->fname . " " . $this->lname;
        }
    }

    // creating class object
    $john = new Person();
    // assigning values to variables
    $john->fname = "John";
    $john->lname = "Wick";
    // calling the method function
    $john->showName();

    echo "<br/>";

    // creating class object
    $james = new Person();
    // assigning values to variables
    $james->fname = "James";
    $james->lname = "Bond";
    // calling the method function
    $james->showName();
?>
```

Output:

```
OUTPUT:

My name is: John Wick

My name is: James Bond
```

The class variables `fname` and `lname`, both are available as separate copies in the objects `$john` and `$james`, where we can assign values to them and then use them.

PHP ACCESS MODIFIERS

To set the access rights for class methods and variables we use access modifiers which are nothing but PHP keywords. We can even assign some of these access modifiers to the class itself to make the class behave in a special way.

Following are the PHP **keywords** which are used as access modifiers along with their meaning:

1. **Public**: When we define class members as public, then they are accessible from anywhere, even from outside of the class scope.
2. **Private:** Private class members can only be accessed from within the class itself.
3. **Protected:** This is same as private, with one exception, the class members defined as protected can still be accessed from its subclass (We will learn about subclasses when we will learn about Inheritance).
4. **Abstract:** This keyword is only used for PHP classes and its member functions.
5. **Final:** The class methods defined as final, cannot be changed or overridden by any subclass.

PUBLIC ACCESS MODIFIER

If we do not specify any access modifiers, all classes and its members are treated as public by default. Public, private or protected access modifiers cannot be used with class. Let's see what happens if we do,

```php
<?php
    public class Studytonight {
        ...
    }
?>
```

OUTPUT:

```
Parse error: syntax error, unexpected 'public' (T_PUBLIC) in ...
```

We will get the above error.

But for class methods and variables we should specify the access specifiers although by default they are treated as public.

This is a simple example of a PHP class:

```php
<?php
    class Studytonight {
        // to store name of person
        var $url = "studytonight.com";

        // simple class method
        function desc() {
            echo "Studytonight helps you learn Coding.";
        }
    }
?>
```

In the code above, we have used the keyword var before the class variable. If we do not use var, we will get a parse error.

But instead of using var we can also use access modifier keywords before the class variable declaration, for example:

```php
<?php
    class Studytonight {
        // to store name of person
        public $url = "studytonight.com";

        // simple class method
        function desc() {
            echo "Studytonight helps you learn Coding.";
        }
    }
?>
```

This is how we should create a PHP class, it is good programming practice to specify the access modifiers along with class variables and methods.

PRIVATE ACCESS MODIFIER

We can use the private access modifier for class variables and methods but not for PHP class. When a class member - a variable or a function, is declared as private then it cannot be accessed directly using the object of the class. For example:

```
<?php
C:\xampp2\htdocs\KAROTA\views\test.html
        // first name of person
        private $fname;
        // last name of person
        private $lname;

        // public function to set value for fname
        public function setFName($fname) {
            $this->fname = $fname;
        }

        // public function to set value for lname
        public function setLName($lname) {
            $this->lname = $lname;
        }

        // public function to
        public function showName() {
            echo "My name is: " . $this->fname . " " . $this->lname;
        }
    }

    // creating class object
    $john = new Person();

    // trying to access private class variables
    $john->fname = "John";  // invalid
    $john->lname = "Wick";  // invalid

    // calling the public function to set fname and lname
    $john->setFName("John");
    $john->setLName("Wick");

?>
```

In the above code, `$fname` and `$lname` are private class variables, hence we cannot directly access them using the class object.

So, when we try to execute the following line of code:

```php
<?php
    $john->setFName("John");
?>
```

We will get a fatal PHP error:

```
OUTPUT:

Fatal error:  Cannot access private property Person::$fname in ...
```

But we can easily access the private variables of a class by defining public functions in the class.

We can create separate functions to set value to private variables and to get their values too.

These functions are called Getters and Setters

```php
class Person {
    // first name of person
    2 references
    private $name;

    // public function to set value for name (setter method)
    1 reference | 0 overrides
    public function setName($name) {
        $this->name = $name;
    }

    // public function to get value of name (getter method)
    1 reference | 0 overrides
    public function getName() {
        return $this->name;
    }
}

// creating class object
$john = new Person();

// calling the public function to set fname
$john->setName("John Wick");

// getting the value of the name variable
echo "My name is " . $john->getName();
```

OUTPUT:

```
My name is John Wick
```

We should have getter and setter methods for all the private variables in the class.

PROTECTED ACCESS MODIFIER

Just like the private access modifier, protected access modifer also restricts the access of class variables and methods outside the class. But the protected class variables and functions can be accessed inside the class and inside the subclass(a class that inherits the class).

We will learn how to create a subclass and about the concept on Inheritance in the upcoming tutorials.

Let's take a quick and simple example:

```php
class Human {
    // protected variable
    1 reference
    protected $genders = array("Male", "Female", "Other");
    // protected function for humans features
    2 references | 0 overrides
    protected function getFeatures($gender) {
        if($gender == "Male") {
            echo "Men will be Men";
        }else if($gender == "Female") {
            echo "Women's mind is a maze.";
        } else if($gender == "Other") {
            echo "All are born equal.";
        }
    }
}
// subclass of the above class
1 reference | 0 implementations
class Male extends Human {
    2 references
    protected $gender = "Male";
    // public function to print Male features
    1 reference | 0 overrides
    public function getMaleFeatures() {
        // calling the protected getFeatures() method of class Human
        $this->getFeatures($this->gender);
    }
}
// object of Human class
$human = new Human();
// object of Male class
$male = new Male();
/*
    accessing protected variables and methods
*/
echo $human->genders;   // INVALID
$human->getFeatures("Male");    // INVALID
echo $male->gender;     // INVALID

/*
    but we can call getMaleFeatures method,
    in which we are calling a protected method
    of Human class
*/
$male->getMaleFeatures();
```

In the program above, we have defined two classes, Human and Male. The class Male is a subclass of the Human class.In the Human class all the class variables and methods are protected, hence they cannot be accessed from outside the class, but they can be accessed inside the subclass of the class Human.

ABSTRACT ACCESS MODIFIER

The abstract access modifier is used with PHP class and its functions. It cannot be used for class variables. If a class has even a single abstract method. then the class must also be defined as abstract.

Also, PHP doesn't allow instantiating the abstract class i.e. you cannot create object of an abstract class, although these classes can be inherited.

We will learn about this access modifier in detail when we will cover Abstract class and Interfaces.

FINAL ACCESS MODIFIER

When we declare a class as final, using this access modifier, then that class cannot be inherited.

Similarly, when we define a class function as final, PHP restricts the subclasses of that class from overriding that function which is declared as final.

Again, we will explain this with help of examples when we will learn about Inheritance.

PHP $this Keyword

this keyword is used inside a class, generally within the member functions to access non-static members of a class(variables or functions) for the current object.

Let's take an example to understand the usage of `$this`.

```php
<?php
    class Person {
        // first name of person
        private $name;

        // public function to set value for name (setter method)
        public function setName($name) {
            $this->name = $name;
        }

        // public function to get value of name (getter method)
        public function getName() {
            return $this->name;
        }
    }

    // creating class object
    $john = new Person();
```

OUTPUT:

```
My name is John Wick
```

In the program above, we have created a private variable in the class with name $name and we have two public methods setName() and getName() to assign a new value to $name variable and to get its value respectively.

Whenever we want to call any variable of class from inside a member function, we use $this to point to the current object which holds the variable.

We can also use $this to call one member function of a class inside another member function.

NOTE: If there is any static member function or variable in the class, we cannot refer it using the $this.

Using self for static Class Members

Instead of $this, for static class members(variables or functions), we use self, along with scope resolution operator ::. Let's take an example,

```php
<?php
    class Person {
        // first name of person
        public static $name;

        // public function to get value of name (getter method)
        public static function getName() {
            return self::$name;     // using self here
        }
    }

?>
```

Difference between PHP self and this

Let's understand a few differences between self and this:

self	this
self keyword is not preceded by any symbol.	this keyword should be preceded with a $ symbol.
To access class variables and methods using the self keyword, we use the scope resolution operator ::	In case of this operator, we use the -< symbol.
It is used to refer the static members of the class.	It is used to access non-static members of the class.
PHP self refers to the class members, but not for any particular object. This is because the static members(variables or functions) are class members shared by all the objecxts of the class.	Whereas, $this wil refer the member variables and function for a particular instance.

Let's take a code example to understand this better:

```php
<?php
    class Job {
        // opening for position
        public $name;
        // description for the job;
        public $desc;
        // company name - as the company name stays the same
        public static $company;

        // public function to get job name
        public function getName() {
            return $this->name;
        }

        // public function to get job description
        public function getDesc() {
            return $this->desc;
        }
```

```
OUTPUT:
Job Name: Data Scientist
Job Description: You must know Data Science
Company Name: Studytonight
```

In the code snippet above we have a few non-static variables and one `static` variable.

Because the `static` members are associated with class itself and not the objects of the class, hence we call them using the class name.

Also, a `static` member function can use a `static` variable inside it, while if a **non-static** method use a `static` variable inside it, then it is also called using the class name, just like a `static` method.

PHP Constructor and Destructor

When we create an object of any class, we need to set properties of that object before using it. We can do that by first initialising the object and then setting values for the properties, either by using the -> operator if the variables are public, or using the public setter methods for the private variables.

To create and initialize a class object in a single step, PHP provides a special method called as Constructor, which is used to construct the object by assigning the required property values while creating the object and for destroying the object **Destructor** method is used.

Syntax for defining Constructor and Destructor

In PHP, we have special functions to define constructor and destructor for a class, they are: __construct() and __destruct().

```php
<?php
    class <CLASS_NAME> {

        // constructor
        function __construct() {
            // initialize the object properties
        }

        // destructor
        function __destruct() {
            // clearing the object reference
        }
    }
?>
```

Constructor can accept arguments, whereas destructors won't have any argument because a destructor's job is to detroy the current object reference.

PHP Constructor

Let's take the example of a class Person which has two properties, fname and lname, for this class we will define a constructor for initialising the class properties(variables) at the time of object creation.

```php
<?php
class Person {
    // first name of person
    private $fname;
    // last name of person
    private $lname;

    // Constructor
    public function __construct($fname, $lname) {
        echo "Initialising the object...<br/>";
        $this->fname = $fname;
        $this->lname = $lname;
    }
    // public method to show name
    public function showName() {
        echo "My name is: " . $this->fname . " " . $this->lname;
    }
}
// creating class object
$john = new Person("John", "Wick");
$john->showName();

?>
```

```
OUTPUT:
Initialising the object...
My name is John Wick
```

While earlier, we were using the -> operator to set value for the variables or used the setter methods, in case of a constructor method, we can assign values to the variables at the time of object creation.

If a class has a constructor then whenever an object of that class is created, the constructor is called.

PHP Constructor Overloading

PHP doesn't support function overloading hence we cannot have multiple implementations for constructor in a class.

PHP Destructor

PHP Destructor method is called just before PHP is about to release any object from its memory. Generally, you can close files, clean up resources etc in the destructor method. Let's take an example,

```php
<?php
    1 reference | 0 implementations
    class Person {
        // first name of person
        2 references
        private $fname;
        // last name of person
        2 references
        private $lname;

        // Constructor
        1 reference | 0 overrides
        public function __construct($fname, $lname) {
            echo "Initialising the object...<br/>";
            $this->fname = $fname;
            $this->lname = $lname;
        }

        // Destructor
        0 references | 0 overrides
        public function __destruct(){
            // clean up resources or do something else
            echo "Destroying Object...";
        }

        // public method to show name
        1 reference | 0 overrides
        public function showName() {
            echo "My name is: " . $this->fname . " " . $this->lname . "<br/>";
        }
    }

    // creating class object
    $john = new Person("John", "Wick");
    $john->showName();

?>
```

OUTPUT:

```
Initialising the object...

My name is: John Wick

Destroying Object...
```

As we can see in the output above, as the PHP program ends, just before it PHP initiates the release of the object created, and hence the destructor method is called.

The destructor method cannot accept any argument and is called just before the object is deleted, which happens either when no reference exist for an object or when the PHP script finishes its execution.

NOTE: If you see a PHP class with a function having same name as the name of the class, then that function will act as the constructor. In older versions of PHP, constructor was not defined using the __construct() name, but it used to have the same name as the class name just like Core Java.

For example:

```
<?php
    1 reference | 0 implementations
    class Person {
        // first name of person
        2 references
        private $fname;
        // last name of person
        2 references
        private $lname;

        // Constructor
        0 references | 0 overrides
        public function Person($fname, $lname) {
            echo "Initialising the object...<br/>";
            $this->fname = $fname;
            $this->lname = $lname;
        }

        // public method to show name
        1 reference | 0 overrides
        public function showName() {
            echo "My name is: " . $this->fname . " " . $this->lname . "<br/>";
        }
    }

    // creating class object
    $john = new Person("John", "Wick");
    $john->showName();

?>
```

PHP INHERITANCE

In object oriented programming, Inheritance enables a class to use properties and methods of an existing class. Often while coding we come across situations where in we have to create a new class with all the functionalities of an existing class and some additional methods, more like an extension to an existing class, in such cases, we can either copy all the properties and methods of the existing class into the new class to make them available in the new class or we can simply inherit the old class in the new class.

Confused? Let's take a simple example to understand it. Consider a class Human with basic methods in it like walk(), eat(), hear(), see() etc. Now if we have to create two more classes for male and female with name Male and Female, with all the properties and methods of the class Human and some specific features available only for Male and Female, we can do so by inheriting Human class in Male and Female class.

The class which is inherited is called Parent class(or super class or base class) while the class which is inheriting other class is called as Child class(or sub class or derived class).

In the example above, Human will be the parent class and Male and Female will be its child classes.

A class can be inherited by multiple classes.

Inheritance is very useful if we want to create several similar classes. We can put the common properties and methods in one parent class and then inherit it in the child classes.

And for destroying the object Destructor method is used.

Syntax for Inheriting a Class

In PHP, extends keyword is used to specify the name of the parent class while defining the child class. For example,

```php
<?php
    class Human {
        // parent class code
    }

    class Male extends Human {
        // child class code
    }

    class Female extends Human {
        // child class code
    }
?>
```

Some important points to remember while using inheritance are:

1. Child class can access and use only the non-private properties and methods on the parent class.
2. Child class can have it's own methods too, which will not be available to the parent class.
3. Child class can also override a method defined in the parent class and provide its own implementation for it.

Let's add a few methods to our Human class and see how we can use them in the child classes Male and Female.

```php
<?php
    // parent class
    2 references | 2 implementations
    class Human {
        // public property name
        4 references
        public $name;
        // public function walk
        1 reference | 0 overrides
        public function walk() {
            echo $this->name. " is walking...<br/>";
        }
        // public function see
        1 reference | 0 overrides
        public function see() {
            echo $this->name. " is seeing...<br/>";
        }
    }
    // child class
    1 reference | 0 implementations
    class Male extends Human {
        // No code in child
    }
      // child class
    1 reference | 0 implementations
    class Female extends Human {
        // No code in child
    }
    $male = new Male();
    $male->name = "Adam";
    $female = new Female();
    $female->name = "Eve";
    // calling Human class methods
    // check the output (hehe, pun intended)
    $female->walk();
    $male->see();

?>
```

OUTPUT:

```
Eve is walking...

Adam is seeing...
```

As you can see from the code above, both the child classes were empty, we only inherited the class Human in both of them, which allowed them to access and use the member properties and methods on the parent class.

Child Class with its Own Methods and Properties

When a child class inherits a parent class, it can access and use all the non-private members of the parent class. We know that, but can a child class has its own member properties and methods? Yes, it can have. Let's take another example to see how we can do this:

```php
<?php
    // parent class
    2 references | 2 implementations
    class Vehicle {
        // public property name
        4 references
        public $name;
        // public function start
        1 reference | 0 overrides
        public function start() {
            echo $this->name. " - Engine start...<br/>";
        }
        // publ
        0 referenc (keyword) function
        public function stop() {
            echo $this->name. " - Engine stop...<br/>";
        }
    }
    // child class
    1 reference | 0 implementations
    class Car extends Vehicle {
        1 reference | 0 overrides
        public function drive() {
            echo "I am " . $this->name . "<br/>";
            echo "Lets go on a drive...";
        }
    }
    // child class
    0 references | 0 implementations
    class Motorcycle extends Vehicle {
        // motorcycle specific properties
        // and methods
    }
    $car = new Car();
    $car->name = "Mercedes benz";
    // calling parent class method
    $car->start();

    // calling child class method
    $car->drive();

?>
```

```
Output:

Mercedes benz - Engine start...

I am Mercedes benz

Lets go on a drive...
```

When a class inherits another class, it gets an advantage of being able to use the properties and methods defined in the parent class without again defining them. And can also have its own properties and methods just like any other normal class.

The protected Access Modifier

We learned about the various access modifiers and how we can use them to control access for various properties and methods of a class.

When a child class inherits a parent class, it can only access and re-use the non-private properties and methods. But we should not use public access modifiers for the properties, as then the properties can be accessed from outside the class too.

To allow only the child class to access properties and methods of the parent class, we can use the protected access modifier.

When we define any property or method of a class as protected then those properties and methods can only be accessed within the child class which inherits the class.

Let's take an example:

```php
<?php
    // parent class
    1 reference | 1 implementation
    class Vehicle {
        // protected property name
        4 references
        protected $name;
        // public function start
        1 reference | 0 overrides
        public function start() {
            echo $this->name. " - Engine start...<br/>";
        }
        // public function stop
        0 references | 0 overrides
        public function stop() {
            echo $this->name. " - Engine stop...<br/>";
        }
    }
    // child class
    1 reference | 0 implementations
    class Car extends Vehicle {
        1 reference | 0 overrides
        public function drive() {
            // accessing name variable of Car class
            echo "I am " . $this->name . "<br/>";
            echo "Lets go on a drive...";
        }
    }
    $car = new Car();
    $car->name = "Mercedes benz";

    // calling parent class method
    $car->start();

    // calling child class method
    $car->drive();
?>
```

OUTPUT:

```
Mercedes benz - Engine start...

I am Mercedes benz

Lets go on a drive...
```

In the example above we have made the name variable as protected, try running the same code with name as private, you will get the following error:

OUTPUT:

```
Notice: Undefined Property...
```

Overriding Parent class Method

What if a child class wants to use a parent class method but slightly differently? It can do so by overriding the definition of the method defined in the parent class and provide its own definition. This is known as Method Overriding.

Let's take an example to understand this concept:

```php
<?php
    // parent class
    2 references | 2 implementations
    class Vehicle {
        // public property name
        2 references
        public $name;
        // public method
        2 references | 2 overrides
        public function drive() {
            echo "Vehicle class drive method...<br/>";
        }
    }
    // child class
    1 reference | 0 implementations
    class Car extends Vehicle {
        2 references | 0 overrides | prototype
        public function drive() {
            echo "Car class drive method...<br/>";
        }
    }
    // child class
    1 reference | 0 implementations
    class Motorcycle extends Vehicle {
        2 references | 0 overrides | prototype
        public function drive() {
            echo "Motorcycle class drive method...<br/>";
        }
    }
    $car = new Car();
    $car->name = "Mercedes benz";

    // calling child class method
    $car->drive();

    $bike = new Motorcycle();
    $bike->name = "Triumph Tiger";

    // calling child class method
    $bike->drive();

?>
```

OUTPUT:

```
Car class drive method...

Motorcycle class drive method...
```

In the code above, we have a parent class named Vehicle and two child classes extending the parent class namely Car and Motorcycle. In the parent class we have a method drive() which we have overriden in the child classes and have provided different definition to it. What if you do not want any child class to override the parent class method? If we want so, we can define the method in the parent class as final.

Let's see what will happen if we will try to override a final method.

```php
<?php
    // parent class
    1 reference | 1 implementation
    class Vehicle {
        // public property name
        1 reference
        public $name;

        // public method
        1 reference
        final public function drive() {
            echo "Vehicle class drive method...<br/>";
        }
    }

    // child class
    1 reference | 0 implementations
    class Car extends Vehicle {

        1 reference | 0 overrides | prototype
        public function drive() {
            echo "Car class drive method...<br/>";
        }

    }

    $car = new Car();
    $car->name = "Mercedes benz";

    // calling child class method
    $car->drive();

?>
```

OUTPUT:

```
Fatal error: Cannot override final method Vehicle::drive()
```

PHP ABSTRACT CLASS AND METHODS

When can define a class abstract using the abstract keyword. An class which is defined as abstract cannot be instantiated.

Following are some important points about abstract class and method:

1. An abstract class can have methods and properties just like any other normal class.
2. An abstract class cannot be instantiated, hence we need to create a child class which extends it, then we can create object of the child class.
3. If a class has even a single abstract method then the class should also be abstract.
4. An abstract method is just the declaration, where we provide name of the method and argument, while the body part is empty.

Don't worry if its too much for you to understand. We will cover all the points step by step with examples, lets start by understanding how we create an abstract class.

Creating an abstract Class

To declare a class abstract, we need to use the abstract keyword before the name of the class.

Let's take an example:

```php
<?php
    // abstract class
    abstract class Vehicle {

        // abstract function mileage
        abstract public function mileage() {

        }
    }

?>
```

In the example above, our class `Vehicle` is an abstract class, which has an abstract method.

The idea behind creating abstract class is to bound developers to follow a set of guidelines, for example, if you want to create a new class which extends our class `Vehicle` then you will have to provide definition for the abstract method `mileage()`, else the child class should also be abstract. Hence, it is mandatory for all the child classes to provide definition for the method `mileage()`.

Non Abstract Method in Abstract Class

Any class with even a single abstract method must be declared abstract. But an abstract class can have non-abstract methods as well, which can be accessed and used directly by the child classes, without overriding them.

Let's extend the example above, and include a non-abstract method in our class Vehicle:

```php
<?php
    // abstract class
    0 references | 0 implementations
    abstract class Vehicle {
        // protected variable
        3 references
        protected $name;
        // non-abstract public function start
        0 references | 0 overrides
        public function start() {
            echo $this->name. " - Engine start...<br/>";
        }
        // non-abstract public function stop
        0 references | 0 overrides
        public function stop() {
            echo $this->name. " - Engine stop...<br/>";
        }
        // non-abstract public function setName
        0 references | 0 overrides
        public function setName($name) {
            $this->name = $name;
        }
        // abstract function mileage
        0 references | 0 overrides
        abstract public function mileage() {
        }
    }


?>
```

In the code above, we have added three non-abstract methods namely start(), stop() and setName() to our abstract Vehicle class.

Inheriting Abstract Classes

Just like any other class, we can create classes extending abstract classes too.

The only difference here is that the child class must provide definition for the abstract method declared in the abstract parent class.

If the child class doesn't provide definition for the abstract method, then it should also be defined as an abstract class.

Let's create two child classes inheriting the class Vehicle and which will have definition for the abstract method mileage():

```php
<?php
    // child class
    class Car extends Vehicle {

        public function mileage() {
            echo "I am " . $this->name . "<br/>";
            echo "My mileage range is - 15 to 22 Km/L";
        }

    }

?>
```

We can have as many child classes as we want, lets have another class:

```php
<?php

    // child class
    class Motorcycle extends Vehicle {

        public function mileage() {
            echo "I am " . $this->name . "<br/>";
            echo "My mileage range is - 35 to 47 Km/L";
        }

    }

?>
```

As mentioned above that an abstract class cannot have any object, once we have proper child classes defined, we can create object for them.

```php
<?php

    $car = new Car();
    $car->setName("BMW X1");
    $car->mileage();

?>
```

```
OUTPUT:
I am BMW X1
My mileage range is - 15 to 22 Km/L
```

If you want to try, go ahead and try to create object of the class Vehicle, you will get an error.

PHP INTERFACE

Just like an abstract class, interface is also created to define the blueprint for classes which inherit the interface. An interface doesn't have abstract methods but public methods with no definition, and the classes inheriting the interface must provide definition to the methods declare inside the interface.

Syntax for defining PHP Interface

Like classes are defined using the keyword class, interfaces are defined using the keyword interface followed by the interface name

```php
<?php
    // interface declaration
    interface NameOfInterface {

    }

?>
```

And a class uses the implements keyword to inherit from an interface and implement the methods declared in the interface.

```php
<?php
    // class declaration
    class SomeClass implements NameOfInterface {

    }

?>
```

Time for an Example

Let's take a simple example where we will create an interface with some methods declared in it and the class which will implement it will be bound to provide definition for those methods.

Following is our interface:

```php
<?php
    // interface declaration
    interface WebApp {

        // methods declaration
        public function login($email, $password);

        public function register($email, $password, $username);

        public function logout();

    }

?>
```

We have defined an interface with name WebApp which has three abstract methods declared in it, namely login(), register() and logout(). As you can see in the code above, we have also provided the parameters that the methods will accept.

Now let's create a class which will implement the above interface:

```php
<?php
    // class declaration
    class Studytonight implements WebApp {

        // methods definition
        public function login($email, $password) {
            echo "Login the user with email: " . $email;
        }

        public function register($email, $password, $username) {
            echo "User registered: Email=".$email." and Username=".$username;
        }

        public function logout() {
            echo "User logged out!";
        }

    }

?>
```

In the class above we have implemented the interface WebApp and provided definition for all the methods declared in the interface.

Implementing Multiple Interface

A class can also implement more than one interface. In that case the class will have to provide definition for the methods declared in all the interfaces implemented by the class.

Let's create another interface:

```php
<?php
    // interface declaration
    interface CMS {

        // methods declaration
        public function publishPost($post);

    }

?>
```

Now let's add the above interface too, to our class Studytonight:

```php
<?php
    // class declaration
    0 references | 0 implementations
    class Studytonight implements WebApp, CMS {

        // methods definition
        0 references | 0 overrides
        public function login($email, $password) {
            echo "Login the user with email: " . $email;
        }

        0 references | 0 overrides
        public function register($email, $password, $username) {
            echo "User registered: Email=".$email." and Username=".$username;
        }

        0 references | 0 overrides
        public function logout() {
            echo "User logged out!";
        }

        0 references | 0 overrides
        public function publishPost($post) {
            echo $post." published!";
        }

    }

?>
```

Now our class Studytonight implements two interfaces.

Some important points to notice:

- All the methods declared in the interfaces are public and do not start with the abstract keyword.
- If we miss to implement even a single method declared in the interface, in the class implementing the interface, we will get an error.
- Interfaces do not have variables.

Don't worry if its too much for you to understand. We will cover all the points step by step with examples, lets start by understanding how we create an abstract class.

Difference between Abstract Class and Interface

Following are a few important differences between an abstract class and an interface:

Interface	Abstract Class
An interface cannot have concrete methods in it i.e. methods with definition.	An abstract class can have both `abstract` methods and concrete methods in it.
All methods declared in interface should be `public`	An abstract class can have `public`, `private` and `protected` etc methods.
Multiple interfaces can be implemented by one class.	One class can extend only one abstract class.

OOP BEST PRACTICES AND DESIGN PATTERNS

A design pattern is a generic reusable object-oriented solution to commonly occurring programming problems that we encounter as developers. Note that design patterns are not modules or libraries that we can simply use in our systems. It is a ready-made template or pattern that can be applied to your designs to solve common problems. It helps you create maintainable and flexible object-oriented systems.

Why should I learn Design Patterns?

- It will let you find a way to solve issues related to software development using a proven solution
- It helps a lot in communication between developers that are familiar to design patterns
- It will improve your object-oriented skills and understanding

OOP Best Practices:

1. **Encapsulation:** Keep data private and provide public methods to access and modify it. This prevents unauthorized access and maintains the integrity of the object's state.

```php
class Car {
    private $model;

    public function setModel($model) {
        $this->model = $model;
    }

    public function getModel() {
        return $this->model;
    }
}
```

2. **Inheritance:** Use inheritance to create a hierarchy of classes where child classes inherit attributes and methods from parent classes.

```php
class Animal {
    protected $name;

    public function __construct($name) {
        $this->name = $name;
    }
}

class Dog extends Animal {
    public function bark() {
        return "Woof!";
    }
}
```

3. **Polymorphism:** Create interfaces or abstract classes to define common behavior, and then implement them in concrete classes. This allows for flexible and interchangeable use of objects.

```php
interface Shape {
    public function calculateArea();
}

class Circle implements Shape {
    private $radius;

    public function calculateArea() {
        return 3.14 * $this->radius * $this->radius;
    }
}
```

4. **Composition over Inheritance:** Favor composition over inheritance. Instead of creating deep class hierarchies, use composition to build complex objects from smaller, reusable components.

```php
class Engine {
    public function start() {
        // Start the engine
    }
}

class Car {
    private $engine;

    public function __construct() {
        $this->engine = new Engine();
    }

    public function startCar() {
        $this->engine->start();
    }
}
```

DESIGN PATTERNS:

1. **Singleton Pattern:** Ensures a class has only one instance and provides a global point of access to that instance.

```php
class Singleton {
    private static $instance;

    private function __construct() {}

    public static function getInstance() {
        if (!self::$instance) {
            self::$instance = new Singleton();
        }
        return self::$instance;
    }
}
```

2. **Factory Pattern:** Provides an interface for creating objects in a super class but allows subclasses to alter the type of objects that will be created.

```php
interface AnimalFactory {
    public function createAnimal();
}

class DogFactory implements AnimalFactory {
    public function createAnimal() {
        return new Dog();
    }
}
```

3. **Observer Pattern:** Defines a one-to-many relationship between objects, so that when one object changes state, all its dependents are notified and updated automatically.

```php
interface Observer {
    public function update($data);
}

class ConcreteObserver implements Observer {
    public function update($data) {
        // Update logic here
    }
}

class Subject {
    private $observers = [];

    public function attach(Observer $observer) {
        $this->observers[] = $observer;
    }

    public function notify($data) {
        foreach ($this->observers as $observer) {
            $observer->update($data);
        }
    }
}
```

These are just a few OOP best practices and design patterns in PHP. Implementing these practices and patterns can help you create more organized, maintainable, and flexible code. Remember that the choice of which pattern to use depends on the specific problem you're trying to solve.